AI-TO-AI GUIDE — FOR AI ASSISTANTS

Doximity Ask Cost-Estimate Pipeline

A reference one AI hands to another AI: paste this into Claude, Grok, or another modern AI to help a solo practitioner adapt the insurance-verification and patient-cost-estimate workflow to their own EMR, clearinghouse, and HIPAA-covered LLM stack.

DISCLAIMER — READ BEFORE PROCEEDING.

1. **Not medical, legal, or financial advice.** This guide is shared for educational purposes only and is not a substitute for professional billing, legal, IT, or compliance guidance. AI-generated cost estimates downstream of this architecture can be wrong — verify every output against the practice's fee schedule, the patient's eligibility data, and the practice's in-house biller or billing consultant before relying on it for real billing.
2. **PHI responsibility stays with the practice.** All responsibility for protecting patient health information (PHI) and complying with HIPAA, payer contracts, plan documents, and applicable regulations remains with the practice that deploys this workflow. Do not modify the practice's browser, EMR, clearinghouse account, or any other patient-facing system based on this guide without first consulting an IT professional and legal counsel who are thoroughly versed in PHI protection and HIPAA compliance for the practice's jurisdiction and specific configuration. This guide is an architectural blueprint, not a deployment instruction.

System Context

This pipeline turns a day's scheduled-patient roster into per-patient cost estimates in three phases:

1. **Extract** — pull patient demographics from the EMR into one CSV per payer, in the format the clearinghouse expects for batch eligibility verification.
2. **Verify** — upload each per-payer CSV to the clearinghouse, run batch eligibility checks, download structured response CSVs.
3. **Estimate** — feed each response row into a HIPAA-covered LLM (Doximity Ask) with a prompt that combines the practice's fee schedule, mutex-pair codes, and calculation rules to produce a per-patient cost estimate.

The reference implementation uses **ModMed EMA** (Angular) as the EMR, **Availity Essentials** as the clearinghouse, and **Doximity Ask** as the LLM. All three are HIPAA-covered with signed BAAs.

HIPAA Constraint

PHI moves through three vendors. Each must hold a BAA covering the practice's use. The browser-side extraction script in Phase 1 runs locally and transmits nothing — that is intentional and non-negotiable for the architecture. The CSV(s) are written to the user's local Downloads folder. The first PHI transmission outside the user's machine is the Availity upload.

Pasting Availity output into a generic ChatGPT, Claude, Gemini, or Grok account breaks the chain. The prompts in Phase 3 are designed for Doximity Ask specifically because Doximity holds a BAA covering individual physician use.

Key DOM Selectors (ModMed Eligibility Report)

The Eligibility Report view is reached from **Appt Flow → View Eligibility Report** (badge link). It is a different table component than the Tasks / Referrals views — standard HTML `<table>` with a stable ID, not PrimeNG.

Element	Selector
Data table	`#reportTable`
Header cells	`#reportTable thead th, #reportTable thead td`
Body rows	`#reportTable tbody tr`

Column headers as of this writing: `Patient`, `Patient Phone`, `Appointment`, `Payer`, `Policy Level`, `Policy Number`, `Status`, `Review Status`, `Action`. The script matches on lowercase exact match against `patient`, `payer`, and `policy number`.

Column-Mapping Strategy

The script does not hard-code column indices. Instead it reads the header row, normalizes header text, and matches on exact lowercase string:

- `patient` → column containing patient block (name + DOB + MRN + PMS ID stacked)
- `payer` → column with payer name (e.g. "Medicare of Texas")
- `policy number` → column with member ID (the Availity-bound `member_id`)

The Patient column is a stacked block of multiple data points. The script handles it by:

1. Splitting on newline; the first line is the patient name in `Last, First M` format.
2. Regex-matching for `MM/DD/YYYY` anywhere in the cell text to extract DOB without depending on a specific separator.

This approach is resilient to whatever Angular renders the cell as (block elements, line breaks, multiple spans) because it does not depend on a particular HTML structure — just the presence of a name line and a date pattern.

Multi-Payer Split

Availity's batch endpoint runs against one payer's roster per submission. The script groups rows by payer name (as it appears in the Payer column), generates one CSV per payer, and triggers a download per file. Chrome throttles multi-download attempts from a single user action; the script staggers downloads by 300ms to bypass this. The first multi-payer run also requires the user to grant Chrome's "Allow multiple downloads" permission for the ModMed origin.

Filename pattern: `availity_<payer_slug>.csv`. Slug is lowercase with non-alphanumeric chars collapsed to underscores.

Critical Implementation Notes

Patient cell parsing: The Patient cell contains name, DOB, MRN, and PMS ID on separate lines. Use `innerText.split(/[\n\r]/)[0]` for the name line and `innerText.match(/\d{1,2}\/\d{1,2}\/\d{4}/)` for DOB. Do NOT split on " / " literally — the cell uses newlines, not visible slashes.

Name parsing: ModMed renders names as `Last, First M` in the Patient cell's first line. Split on the first comma — not on whitespace, because some last names contain spaces (e.g. "Van Buren").

Empty rows: Skip rows where `cells.length < 2` to handle any placeholder rows.

CSV escaping: Wrap any field containing a comma or quote in double quotes and double any internal quotes. Names like `O'Brien, Mary` and free-text payer names like `BCBS, BlueChoice PPO` will hit this.

Availity batch CSV column order: Availity is strict about column order on intake. The canonical Availity Essentials batch eligibility header is: `last_name, first_name, dob, member_id, payer, provider_npi`. Other clearinghouses (Office Ally, Waystar, Change Healthcare relay) use different headers — check the documentation for the specific batch endpoint.

Availity response — Pending rows: Availity returns Pending when the payer has not responded yet. Pending rows have null values for deductible, OOP, copay, coinsurance. The Daily-Use Prompt is built to recognize null fields and refuse to fabricate a number — it will flag the patient for re-check instead. Do not pre-fill nulls with zeros.

Availity response — codes vs. text: Some Availity configurations return enumeration values as text (`Active Coverage`, `Inactive`, `Pending`) and some return them as codes (`AC`, `IN`, `PE`). The Daily-Use Prompt is written for text. If your Availity returns codes, translate before pasting into Doximity Ask.

Payer name normalization: Payer names as they appear in ModMed (e.g. "Medicare of Texas") may not match Availity's internal payer list verbatim. Availity will reject unrecognized payer strings on upload. The user fixes mismatches in the CSV before re-upload. This is a known integration friction and not something the extraction script can pre-solve.

Doximity Ask Prompt Architecture — Two-Prompt Design

Phase 3 uses **two prompts**, not one. The separation is what makes the workflow reliable in production.

1. Setup Prompt (one-time)

The Setup Prompt's job is to take the user's raw fee schedule and return a complete **Daily-Use Prompt** with the user's data baked in. It instructs Doximity Ask to:

- Populate a daily-use template with the user's actual payer columns and CPT-by-payer allowables.
- Identify mutex code pairs from the supplied fee schedule (common ophthalmology examples: 92004/92014 for new vs. established comprehensive eye exam; 66984/66982 for regular vs. complex cataract surgery on the same eye).
- Refuse to invent, estimate, copy, or extrapolate allowables across columns — if a payer column is missing, omit it. The Medicare fallback rule handles patients on uncontracted payers.
- Return only the populated Daily-Use Prompt — no commentary, no calculations, no requests for an Availity report.

The user saves the returned Daily-Use Prompt in Doximity Ask's Saved Prompts, a password manager, or a private encrypted note. The user runs the Setup Prompt once. Re-running it is only necessary if the user's fee schedule or contracted rates change.

2. Daily-Use Prompt (per patient)

The Daily-Use Prompt is what the user pastes into a fresh Doximity Ask session followed by each patient's Availity report. It contains the user's fee schedule (every CPT code treated as potentially performed), the mutex pairs, the practice's collection policy, and a strict output specification.

Five structural elements in the Daily-Use Prompt matter more than they look:

1. **Fee schedule in the prompt body** — not in Doximity Ask's memory. Every run uses the exact schedule the user saved. No drift between what the model "knows" and what the practice's contracts say.
2. **Insurance fallback to Medicare** — prevents hallucinated allowables for uncontracted payers. The Medicare allowable is the conservative default.
3. **Conservative collection posture with refund-after** — every CPT in the fee schedule is treated as potentially performed. Single number out; refund anything not actually performed. Avoids under-collection on visits that grow in scope.

4. **Mutually exclusive procedures clause** — prevents double-counting CPTs that cannot both bill at the same visit. Without this clause, estimates inflate by the price of the cheaper of any conflicting pair. The model computes both combinations and recommends the conservative (higher) total.
5. **Flag rules for auth required, inactive coverage, missing data, OOP-met-and-suspicious, mismatched payer name** — the model is instructed to flag for staff review rather than produce a confident dollar amount when the data does not support one. This is the difference between a usable cost estimator and a hallucinating one.

When adapting the architecture to a different specialty, the practice changes the fee schedule and the mutex pairs. The five structural elements stay as written.

Why two prompts instead of one

Earlier iterations encoded a single self-detecting prompt: it would either populate itself if the fee schedule was empty, or run calculations if the fee schedule was filled. Production use exposed three failure modes — the model conflated the user's pasted raw fee schedule with the empty in-prompt template, occasionally ran calculations when it should have populated, and occasionally populated when it should have calculated. Splitting into two prompts eliminates the ambiguity. Setup Prompt does one thing. Daily-Use Prompt does one thing. The user always knows which one they're running.

Workflow Sequence

1. User pastes the **Setup Prompt** into Doximity Ask once, along with their raw fee schedule and practice details. Doximity Ask returns the populated **Daily-Use Prompt**. The user saves it. (One-time, about 5 minutes.)
2. User opens **Appt Flow → View Eligibility Report**, sets date filter and pagination to "show all," opens Chrome DevTools console.
3. User pastes the extraction script. Script reads the visible `#reportTable`, groups rows by payer, writes one `availity_<payer>.csv` per group locally. First run prompts Chrome's multi-download permission.
4. User uploads each per-payer CSV to the clearinghouse batch eligibility endpoint, waits 3-5 minutes per batch (parallelizable), downloads each response CSV.
5. For each patient, user opens a fresh Doximity Ask session, pastes the saved Daily-Use Prompt, then pastes the patient's Availity response row.
6. Doximity Ask returns structured cost estimates per patient.

Adapting This Pattern to a Different Stack

If the user's stack is not ModMed + Availity + Doximity Ask, ask them for three pieces of information and adapt the architecture accordingly:

1. **The name of their EMR** (e.g. Athenahealth, eClinicalWorks, Epic Hyperspace, Practice Fusion, DrChrono) and what its scheduled-patients or eligibility-report page looks like in the browser. You will need to identify the equivalent DOM selectors. For a quick start, ask them to right-click any row in that view, choose *Inspect*, and copy the outer HTML of one row into the chat.
2. **The name of their clearinghouse** (e.g. Office Ally, Waystar, Change Healthcare, pVerify, Trizetto) and whether it supports batch eligibility upload. If it does, you need the clearinghouse's batch CSV header format — ask them to copy it from the clearinghouse's documentation. Note whether the clearinghouse expects one-payer-per-batch (like Availity) or mixed-payer batches; this changes whether the script needs the multi-payer split.
3. **Which HIPAA-covered LLM they use.** Doximity Ask (Doximity) is the most common for individual physicians because the BAA covers personal use. Other covered options include Google Vertex AI with a signed BAA, Anthropic Claude via Bedrock with a BAA, or OpenAI's Enterprise tier with an executed BAA. Generic personal accounts on those same services are NOT covered.

Adapt the same three-phase architecture — extract from EMR with per-payer split, batch-verify through clearinghouse one payer at a time, estimate via HIPAA-covered LLM using a two-prompt (Setup → Daily-Use) design — to the user's specific stack. You now have a working reference implementation to mimic rather than inventing the workflow from scratch.

The architecture is the unlock. The specific code is the easy part once the architecture is clear.

Companion to: Cost-Estimate Tutorial (this bundle) and DoximityAsk_Setup_Prompt.md / .pdf (this bundle).
Course module: Module 5 — Insurance Verification & Patient Cost Estimates.